COL216 - Computer Architecture

Assignment 3

Cache Friendly Graph Processing

Department of Computer Science & Engineering
Indian Institute of Technology, Delhi
Semester II, 2025–26

Report Submitted By:

Agrim Singh 2024CS10071

Contents

1 Overview **3**

2 Implementation **3**
- 2.1 Pointer-Based Adjacency List . . . 3
- 2.2 Compressed Sparse Row (CSR) . . . 3
- 2.3 Implementation Correctness and Memory Safety . . . 4

3 Methodology **4**
- 3.1 Workload Generation . . . 4
- 3.2 Timing and Statistical Isolation . . . 4
- 3.3 Architectural Cache Simulation . . . 5

4 Timing Analysis **5**
- 4.1 Experimental Setup and Statistical Rigor . . . 5
- 4.2 Performance Metrics . . . 6
- 4.3 Architectural Observations . . . 6
- 4.4 Chain Graph With 100,000 Nodes . . . 6
- 4.5 Grid Graph with Dimension 316 x 316 . . . 7
- 4.6 Random Graph With 30,000 Nodes, Average Degree 8 . . . 8
- 4.7 Random Graph With 50,000 Nodes, Average Degree 8 . . . 10
- 4.8 Random Graph With 50,000 Nodes, Average Degree 32 . . . 10
- 4.9 Random Graph With 100,000 Nodes, Average Degree 8 . . . 11
- 4.10 Star Graph With 100,000 Nodes . . . 12

5 Cache Profiling **14**
- 5.1 With Fixed Cache Parameter - Varying Input Graphs . . . 14
- 5.2 Varying Associativity . . . 21
- 5.3 Varying Cache Size . . . 22
- 5.4 Varying Line Size . . . 23

6 Conclusion **24**

1 Overview

Graph algorithms such as Breadth-First Search (BFS) are frequently bottlenecked by memory latency rather than computational complexity. This study explores the performance implications of data layout by comparing two distinct graph representations: a traditional **Pointer-Based Adjacency List** and a **Compressed Sparse Row (CSR)** format.

The primary objective is to quantify how memory locality and hardware interaction influence execution efficiency. While the pointer-based approach offers implementation flexibility, it often suffers from *pointer chasing* and poor spatial locality due to fragmented heap allocations. Conversely, CSR leverages contiguous arrays to minimize pointer overhead and maximize hardware prefetching efficiency.

To evaluate these effects, we implemented BFS for both layouts and utilized `Cachegrind` to profile architectural metrics, including D1 and LLd cache misses, across varying cache sizes, associativities, and line sizes. This investigation demonstrates that algorithmic performance is fundamentally linked to how effectively a data structure aligns with modern CPU cache hierarchies.

2 Implementation

We implemented a standard Breadth-First Search (BFS) algorithm to compute shortest paths from a given source vertex. While the core algorithmic logic remained constant across both versions—utilizing a FIFO queue for vertex exploration—the memory access patterns differed significantly based on the underlying graph representation.

2.1 Pointer-Based Adjacency List

In the pointer-based implementation, each vertex stores a pointer to a linked list of `Edge` objects.

- **Traversal:** Exploring neighbors requires *pointer chasing* where the algorithm dereferences a head pointer and iteratively follows `next` pointers to traverse the list.
- **Memory Characteristics:** Each edge is a separate heap-allocated object, leading to potential fragmentation and poor spatial locality.

2.2 Compressed Sparse Row (CSR)

The CSR representation flattens the graph structure into two contiguous integer arrays to eliminate pointer overhead.

- **Structure:** We implemented a conversion utility that calculates the degree of each vertex to populate a `row_ptr` array (offsets) and a `col_idx` array (neighbor indices).
- **Traversal:** Neighbor exploration is transformed from a linked-list traversal into a bounded loop over a contiguous array slice
- **Efficiency:** This layout leverages sequential memory access, allowing the CPU hardware prefetcher to load edge data more effectively and reducing the frequency of cache misses.

2.3 Implementation Correctness and Memory Safety

Both implementations were verified to return the same number of visited nodes and identical distance arrays. To ensure an accurate performance comparison, we utilized a common queue structure and initialization routine, isolating the graph traversal method as the primary independent variable. Additionally, `malloc` and `free` were employed carefully to ensure safe dynamic memory management. Systematic memory profiling using Valgrind confirmed that all allocated memory was properly reclaimed, with no memory leaks or illegal access patterns detected during execution.

3 Methodology

To rigorously evaluate the performance characteristics of Breadth-First Search (BFS) using traditional pointer-based adjacency lists versus Compressed Sparse Row (CSR) representations, we developed an automated benchmarking pipeline. The testing framework was designed to evaluate both raw execution speed and architectural cache efficiency across various graph topologies. To minimize experimental variance and mitigate interference from the OS scheduler or background processes, **all benchmarking and profiling tasks were pinned to a single CPU core** to ensure serial execution in a noise-isolated environment.

3.1 Workload Generation

To ensure the implementations were tested against both predictable and unpredictable memory access patterns, we generated synthetic graphs across three primary categories:

- **Erdős–Rényi (ER) Random Graphs:** Representing irregular, unpredictable memory topologies. We varied both vertex counts (20,000 to 100,000) and average degrees (sparse $d = 8$ and dense $d = 32$) to evaluate the impact of edge density on cache miss rates.
- **Grid Graphs (e.g., 316×316):** Providing a baseline for highly structured, predictable spatial locality.
- **Chain and Star Graphs:** Serving as boundary test cases to evaluate extreme search depths versus extreme branching factors.

3.2 Timing and Statistical Isolation

Accurately measuring the microsecond-level execution time of a single BFS traversal is challenging due to fixed program overheads, such as memory allocation and graph loading. To address this, we utilized the provided `benchmark.c` infrastructure to calculate the execution times for both pointer-based and CSR-based BFS using a differential statistical method.

For each graph, we executed workloads at varying repetition counts (R_{small} and R_{large}). By applying the formula:

$$\text{Execution Time} = \frac{T_{large} - T_{small}}{R_{large} - R_{small}}$$

we mathematically canceled out constant startup overheads. To further refine our analysis, we implemented a custom component, `phase_timer.c`, which allowed us to isolate the specific latency of the pointer-to-CSR conversion process. This separate timing logic ensured that the high computational cost of rebuilding the graph into a contiguous format did not contaminate the pure traversal measurements of the CSR algorithm.

3.3 Architectural Cache Simulation

Execution time alone obscures the underlying memory bottlenecks inherent to graph traversal. To map performance directly to CPU architecture, we profiled the implementations using Valgrind's Cachegrind tool.

A critical aspect of our profiling strategy was the logical separation of cache metrics. We explicitly isolated the misses generated during the `CSR BFS` phase from those generated during the `CSR Conversion` phase. This separation provided a cleaner dataset for analysis, allowing us to observe how the contiguous memory layout of CSR benefits the search algorithm independently of the one-time overhead required to create that layout.

We performed a parametric sweep of the Level 1 (L1) data cache to determine how hardware configurations favor contiguous data structures (CSR) over fragmented ones (Pointers). The parameters evaluated included:

- **Cache Size:** Sweeping from 16 KB to 64 KB to observe capacity miss reduction.
- **Associativity:** Comparing 2-way versus 4-way versus 8-way sets to evaluate conflict miss handling.
- **Line Size:** Testing 32 B versus 64 B versus 128 B lines to explicitly measure the spatial locality benefits of the CSR format's contiguous edge arrays.

4 Timing Analysis

In this section, we present a comparative performance analysis of BFS execution times across seven distinct input graphs, representing a diverse range of topologies including Chain, Star, Grid, and Erdős–Rényi models. This variety allows us to evaluate how different graph shapes influence the efficiency of pointer-based versus contiguous memory layouts.

4.1 Experimental Setup and Statistical Rigor

To ensure statistical significance and filter out transient system noise, we adopted the following protocol for each test case:

- **Sample Size:** We collected 100 high-fidelity samples for each graph.
- **Warmup Phase:** Each benchmark was preceded by 5 warmup iterations. These samples were discarded to ensure the CPU frequency scaling (DVFS) had stabilized and that the relevant instruction and data caches were *hot* before measurement began.
- **Visualization:** Data processing and visualization were handled via a custom Python suite utilizing the `Matplotlib` library to generate distribution histograms and comparative bar charts.

4.2 Performance Metrics

The primary metric for comparison is the **CSR Speedup**, calculated as the ratio of the mean execution times:

$$\text{CSR Speedup} = \frac{\text{Mean Pointer BFS Time}}{\text{Mean CSR BFS Time}}$$

A speedup factor greater than 1.0 indicates that the CSR implementation outperformed the pointer-based adjacency list.

4.3 Architectural Observations

Across nearly all test cases, the mean execution time for CSR was consistently lower than that of the pointer-based implementation. While the magnitude of this benefit varied—being highly pronounced in dense graphs and more modest in sparse chains—the trend remains clear.

We observe two critical phenomena:

1. **Reduced Latency:** CSR reduces the total number of cycles stalled on memory. By eliminating pointer chasing (dereferencing $Edge \rightarrow next$), CSR allows the hardware prefetcher to anticipate the next neighbor's address within the contiguous `col_idx` array.

2. **Lower Variance:** Our data shows that CSR generally exhibits lower variance in execution time compared to the pointer implementation. This is due to the **deterministic nature of contiguous memory access**. Pointer-based BFS relies on heap-allocated nodes that may be scattered across non-contiguous memory pages; this introduces jitter caused by unpredictable TLB (Translation Lookaside Buffer) misses and varying DRAM row-buffer hits. CSR's linear layout provides a more stable, cache-friendly execution environment.

4.4 Chain Graph With 100,000 Nodes

We observe a CSR speedup of approximately 1.18×, indicating that even in highly linear topologies with minimal branching, the reduction in pointer-chasing overhead provides a measurable efficiency gain.

The pointer-based distribution is significantly more spread out, likely due to the unpredictable latency of traversing linked list nodes scattered across the heap. In contrast, CSR's tighter clustering confirms that contiguous memory access leads to more deterministic performance by maximizing L1 cache hits and streamlining the hardware prefetching process.

Metric	Value
Mean Pointer BFS Time (ms)	0.45009875
Mean CSR BFS Time (ms)	0.38009000
Mean CSR Conversion Time (ms)	0.29409174
Pointer BFS Variance	0.00538753
CSR BFS Variance	0.00178804
CSR Speedup	1.18418993

Table 1: Chain Graph : 100,000 Nodes

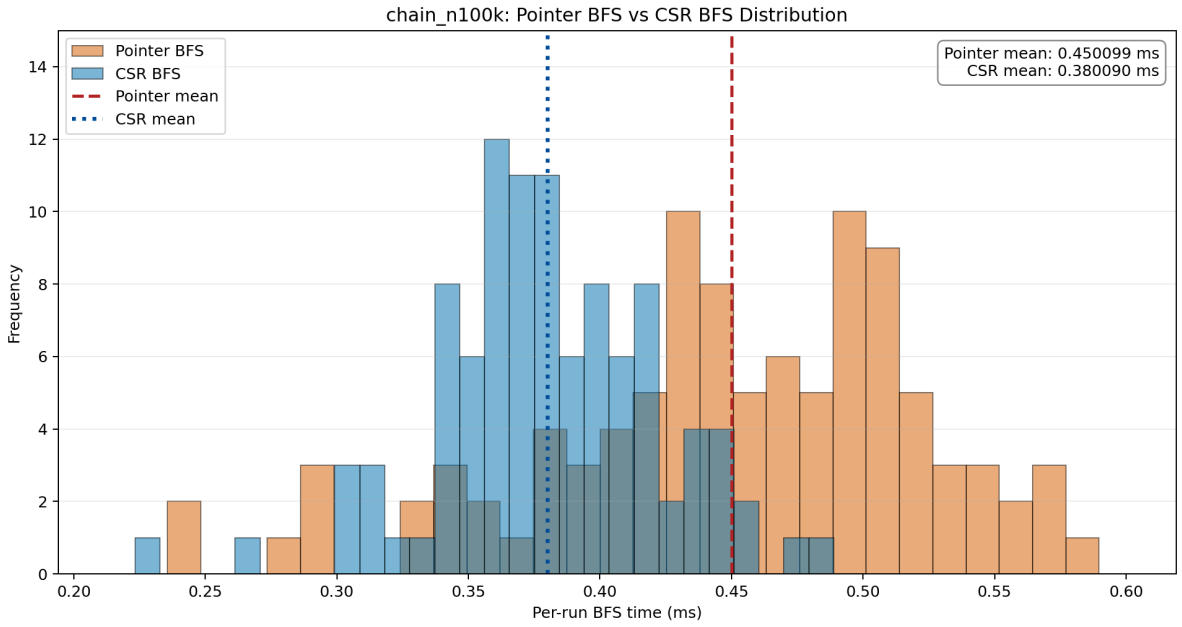


Figure 1: Chain Graph : 100,000 Nodes

4.5 Grid Graph with Dimension 316 x 316

This doubling of efficiency is driven by the increased edge density and structural regularity of the grid, which allows the CSR layout to excel in spatial locality. Because neighbors are stored in a contiguous `col_idx` array, a single cache line can often load multiple adjacent edges simultaneously, whereas the pointer-based version requires separate, high-latency memory fetches for each linked list node.

The CSR distribution is notably more deterministic, exhibiting a variance of only 0.0016 compared to 0.0113 for the pointer implementation. This tight clustering confirms that sequential memory access patterns allow the hardware prefetcher to effectively hide memory latency, whereas the pointer chasing required by the adjacency list leads to unpredictable stalls and higher execution jitter.

Metric	Value
Mean Pointer BFS Time (ms)	0.84429875
Mean CSR BFS Time (ms)	0.38634375
Mean CSR Conversion Time (ms)	0.87209495
Pointer BFS Variance	0.01128762
CSR BFS Variance	0.00161057
CSR Speedup	2.18535631

Table 2: Grid Graph : 316 x 316 (around 100,000 Nodes)

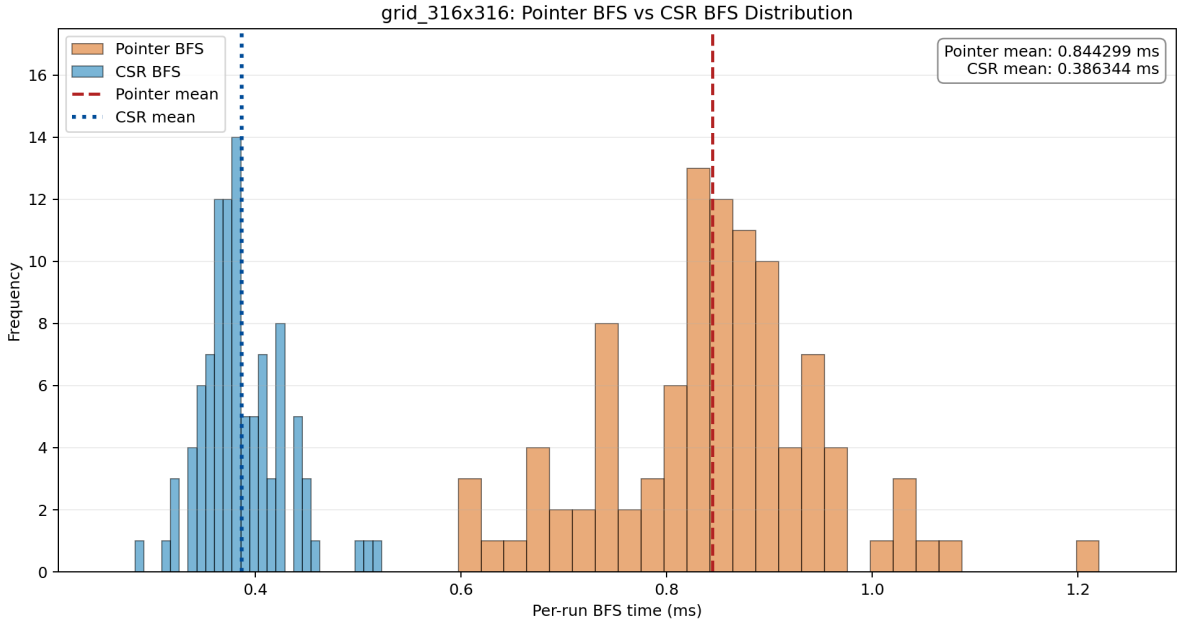


Figure 2: Grid Graph : 316 x 316 (around 100,000 Nodes)

4.6 Random Graph With 30,000 Nodes, Average Degree 8

The CSR implementation achieves a mean BFS time of 0.442 ms, resulting in a 1.651× speedup over the pointer-based approach, which records a mean of 0.730 ms. In addition to improved throughput, CSR demonstrates greater consistency, with a variance of 0.0019—approximately 2.85× lower than the pointer-based variance of 0.0056—indicating that the observed performance gains are stable across runs. The conversion overhead of 0.336 ms is also lower than the time required for a single pointer-based BFS, meaning the cost of conversion is effectively amortized after just one traversal.

These gains are largely explained by memory behavior in random graphs such as Erdős–Rényi models, where neighbor vertices are distributed unpredictably. Pointer-based adjacency lists worsen this by scattering data across the heap, while CSR compacts neighbor indices into a contiguous array, enabling sequential access. This significantly improves cache efficiency: with an average degree of 8, a 64-byte cache line can store most of the adjacency lists for two vertices, allowing multiple edges to be processed per

cache miss—something not possible with fragmented pointer structures. Furthermore, the tighter and left-shifted distribution observed for CSR indicates that hardware prefetching is more effective, as the predictable memory layout allows the CPU to preload upcoming data, unlike the irregular access patterns of pointer chasing, which introduce frequent stalls.

Metric	Value
Mean Pointer BFS Time (ms)	0.73011375
Mean CSR BFS Time (ms)	0.44207250
Mean CSR Conversion Time (ms)	0.33615399
Pointer BFS Variance	0.00569127
CSR BFS Variance	0.00199359
CSR Speedup	1.65157016

Table 3: ER Graph : 30,000 Nodes and Average Degree 8

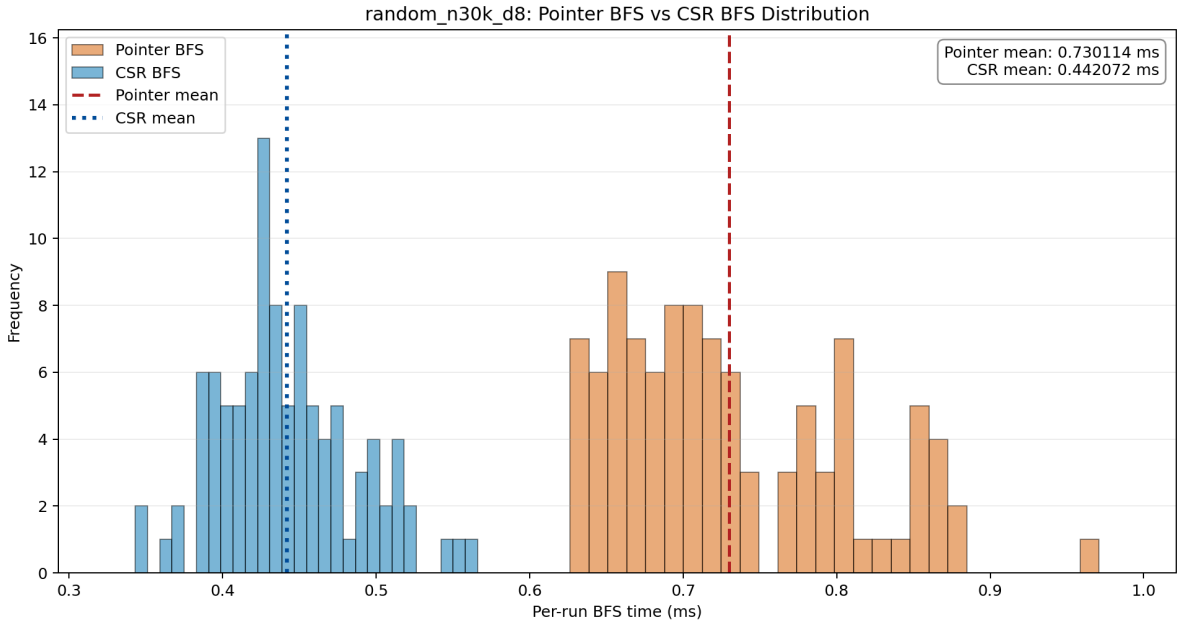


Figure 3: ER Graph : 30,000 Nodes and Average Degree 8

4.7 Random Graph With 50,000 Nodes, Average Degree 8

Metric	Value
Mean Pointer BFS Time (ms)	1.73534650
Mean CSR BFS Time (ms)	0.79798375
Mean CSR Conversion Time (ms)	0.81786903
Pointer BFS Variance	0.02361091
CSR BFS Variance	0.00194475
CSR Speedup	2.17466396

Table 4: ER Graph : 50,000 Nodes and Average Degree 8

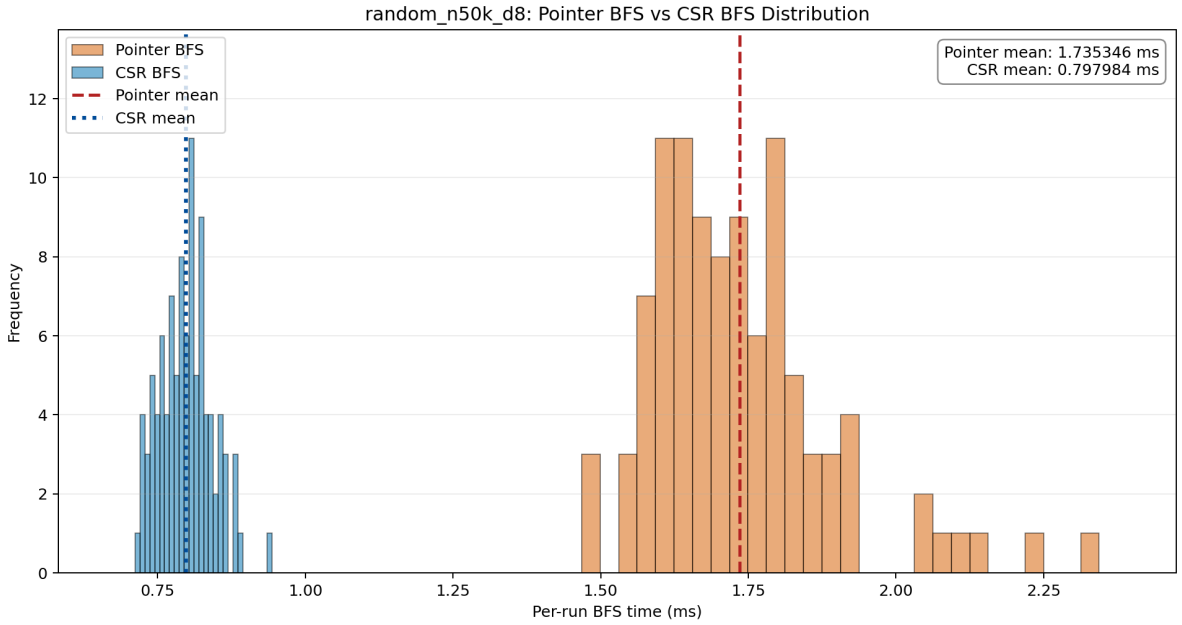


Figure 4: ER Graph : 50,000 Nodes and Average Degree 8

4.8 Random Graph With 50,000 Nodes, Average Degree 32

The CSR representation delivers a mean BFS time of 1.959 ms, producing a substantial 7.04× speedup over the pointer-based implementation, which records a mean of 13.798 ms. This large gap reflects how performance penalties in pointer structures scale aggressively with increasing edge density: as the average degree rises to 32, the number of pointer dereferences per vertex expands significantly, amplifying latency from cache misses and memory indirection. CSR avoids this compounding overhead by storing all adjacency information in a dense linear layout, allowing traversal to proceed with predictable memory strides even under heavy connectivity. The variance difference is also pronounced—0.011 for CSR versus 0.502 for pointer-based BFS—demonstrating that the pointer model becomes increasingly unstable as structural complexity grows, while CSR

maintains tight runtime clustering. Although the conversion cost is 7.487 ms, it remains well below the cumulative cost of repeated pointer traversals in dense graphs, making CSR particularly advantageous in workloads where graph density is high and traversal operations are frequent.

Metric	Value
Mean Pointer BFS Time (ms)	13.79829400
Mean CSR BFS Time (ms)	1.95889540
Mean CSR Conversion Time (ms)	7.48734500
Pointer BFS Variance	0.50237243
CSR BFS Variance	0.01118702
CSR Speedup	7.04391567

Table 5: ER Graph : 50,000 Nodes and Average Degree 32

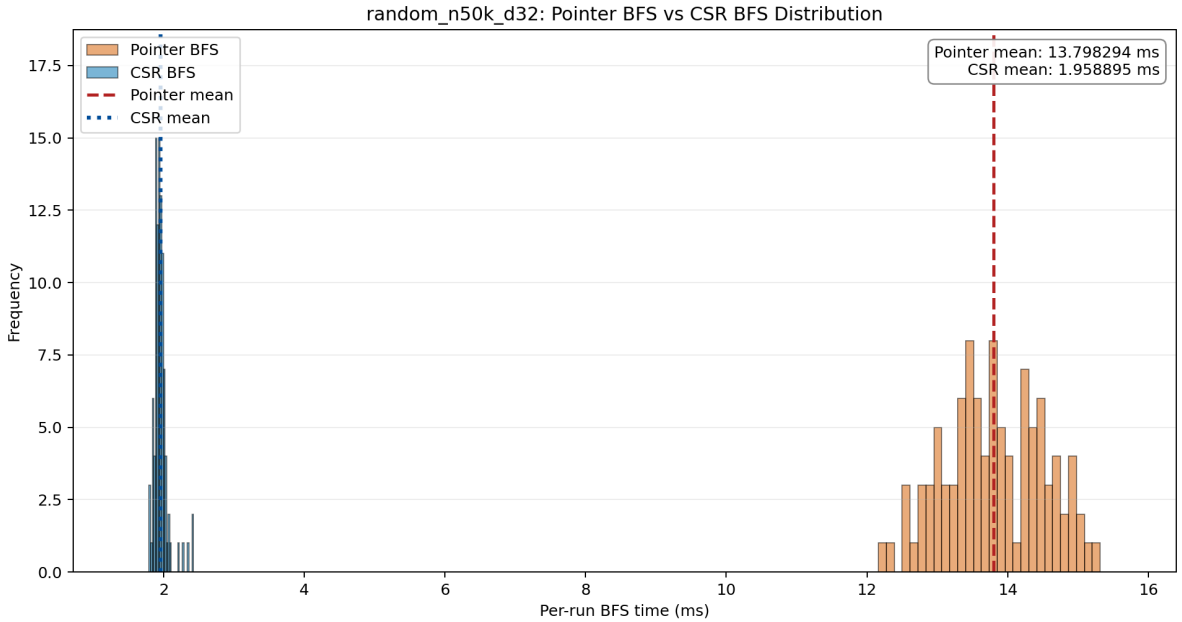


Figure 5: ER Graph : 50,000 Nodes and Average Degree 32

4.9 Random Graph With 100,000 Nodes, Average Degree 8

In this configuration, CSR achieves a mean BFS time of 1.796 ms compared to 5.805 ms for the pointer-based structure, yielding a 3.23× speedup. The improvement is driven less by density and more by scale: doubling the vertex count increases the total working set size, placing greater pressure on memory hierarchy behavior. Pointer-based adjacency lists fragment this larger dataset across many allocations, increasing Translation Lookaside Buffer (TLB) pressure and reducing spatial locality during traversal. CSR consolidates the same edge set into contiguous memory regions, enabling more efficient page utilization and reducing address translation overhead. The stability advantage persists as well, with

CSR variance at 0.0038 compared to 0.1446 for pointer BFS, indicating fewer long-tail latency spikes. Notably, the conversion time of 2.473 ms is only modestly larger than a single CSR traversal, meaning the preprocessing overhead remains bounded even as the graph size grows, preserving the practical viability of CSR for large sparse networks.

Metric	Value
Mean Pointer BFS Time (ms)	5.80488200
Mean CSR BFS Time (ms)	1.79553750
Mean CSR Conversion Time (ms)	2.47296600
Pointer BFS Variance	0.14457315
CSR BFS Variance	0.00375650
CSR Speedup	3.23294946

Table 6: ER Graph : 100,000 Nodes and Average Degree 8

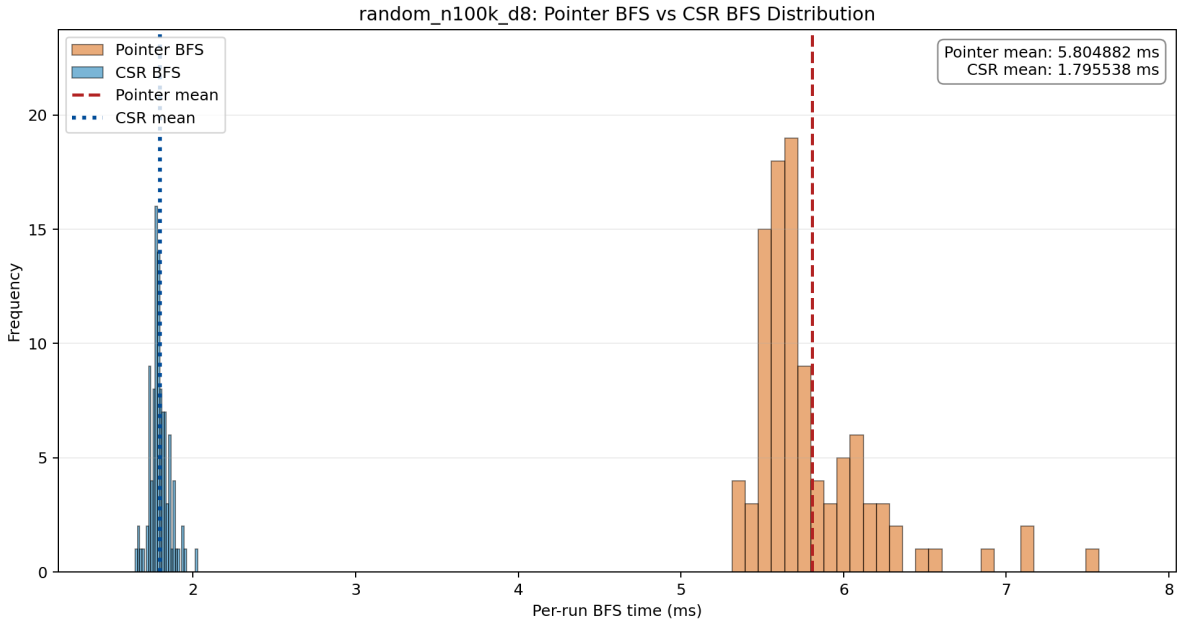


Figure 6: ER Graph : 100,000 Nodes and Average Degree 8

4.10 Star Graph With 100,000 Nodes

For the star topology, CSR records a mean BFS time of 0.132 ms versus 0.226 ms for the pointer-based implementation, resulting in a more modest 1.71× speedup. The reduced margin is expected because the traversal pattern in a star graph is inherently shallow—most vertices are reached in one expansion step—so the total number of memory accesses is limited regardless of representation. In this scenario, the dominant factor becomes the handling of a single high-degree hub node. CSR processes the hub's large neighbor set as a continuous block, minimizing branch overhead and enabling rapid iteration through the edge list, whereas the pointer structure incurs repeated pointer

dereferences during that same burst of activity. The variance values—0.00092 for CSR and 0.00395 for pointer BFS—indicate that CSR still delivers more predictable execution timing, but the absolute difference is smaller because the topology constrains the opportunity for performance divergence. The conversion overhead of 0.522 ms exceeds a single traversal cost, implying that in highly centralized graphs with minimal traversal depth, the benefits of CSR emerge primarily when multiple BFS operations are performed rather than in one-off executions.

Metric	Value
Mean Pointer BFS Time (ms)	0.22587750
Mean CSR BFS Time (ms)	0.13199000
Mean CSR Conversion Time (ms)	0.52159887
Pointer BFS Variance	0.00394785
CSR BFS Variance	0.00091584
CSR Speedup	1.71132283

Table 7: Star Graph : 100,000 Nodes

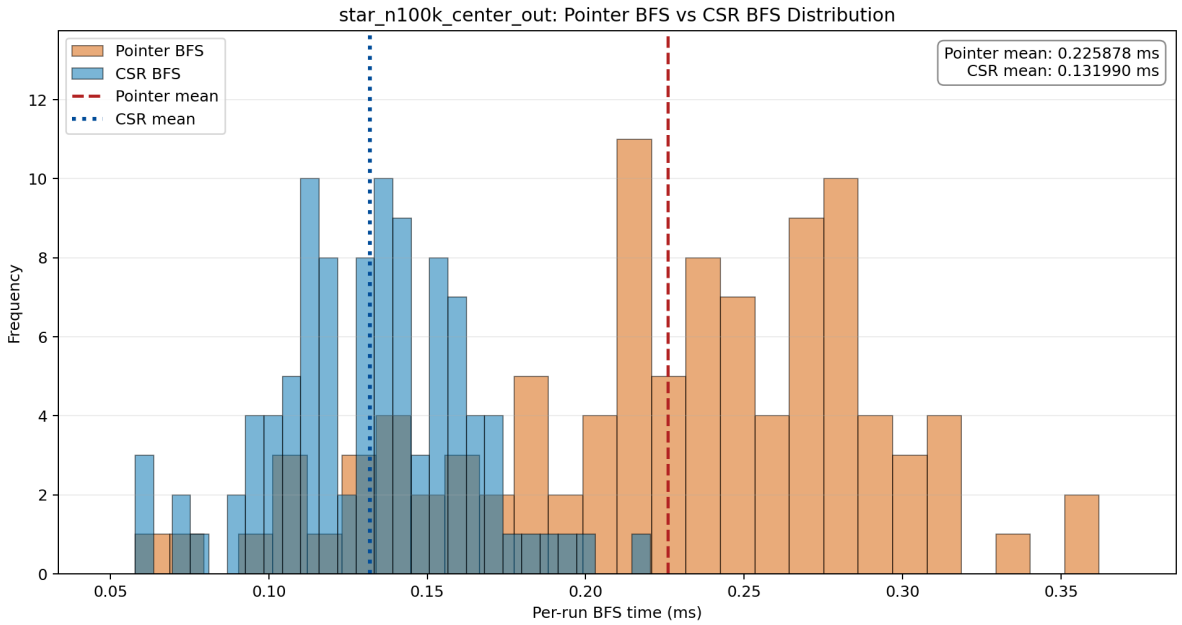


Figure 7: Star Graph : 100,000 Nodes

5 Cache Profiling

5.1 With Fixed Cache Parameter - Varying Input Graphs

Cache Parameters used - 32KB cache, 8-way associative and 64B line size. Input varied across 7 input graphs mentioned earlier. These values of cache misses are for 1 sample.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	2500024	3001756	186538107
Data refs	900011	1400685	69770452
I1 misses	4	16	1696
LLi misses	4	16	1663
I1 miss rate	0.00016000	0.00053302	0.00090920
LLi miss rate	0.00016000	0.00053302	0.00089151
D1 misses	75005	31288	315132
LLd misses	6250	12503	76570
LL misses	6254	12519	78233
D1 miss rate	8.33378703	2.23376419	0.45166971
LLd miss rate	0.69443596	0.89263468	0.10974560
LL miss rate	0.18393928	0.28436497	0.03052298

Table 8: Chain Graph : 100,000 Nodes

Chain Graph (100k nodes)

- **D1: Pointer > CSR** – Linked structures cause pointer chasing with poor spatial locality, whereas CSR uses contiguous memory.
- **LLd: CSR ≥ Pointer** – Predictable streaming keeps LLd misses low for both; CSR fetches slightly more contiguous blocks, so more LLd misses

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	4584561	5383004	559824833
Data refs	1793630	2293192	205306508
I1 misses	4	0	1724
LLi misses	4	0	1690
I1 miss rate	0.00008725	0.00000000	0.00030795
LLi miss rate	0.00008725	0.00000000	0.00030188
D1 misses	422784	297692	1083389
LLd misses	6240	12483	244261
LL misses	6244	12483	245951
D1 miss rate	23.57141662	12.98155584	0.52769345
LLd miss rate	0.34789784	0.54435041	0.11897382
LL miss rate	0.09789610	0.16261961	0.03214494

Table 9: Grid Graph : 316 x 316 (around 100,000 nodes)

Grid Graph (316 × 316)

- **D1: Pointer > CSR** – 2D traversal leads to scattered pointer accesses; CSR packs neighbors more efficiently.
- **LLd: CSR > Pointer** – Larger contiguous accesses in CSR increase deeper cache pressure slightly.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	2219367	2580415	303137679
Data refs	899744	1050145	108424283
I1 misses	4	2	1717
LLi misses	4	2	1684
I1 miss rate	0.00018023	0.00007751	0.00056641
LLi miss rate	0.00018023	0.00007751	0.00055552
D1 misses	395022	281782	637787
LLd misses	1875	3751	142195
LL misses	1879	3753	143879
D1 miss rate	43.90382153	26.83267549	0.58823262
LLd miss rate	0.20839261	0.35718877	0.13114682
LL miss rate	0.06024152	0.10337248	0.03495926

Table 10: ER Graph : 30,000 nodes and average degree 8

ER Graph (30k, deg 8)

- **D1: Pointer > CSR** – Random edges amplify pointer chasing; CSR benefits from compact storage.
- **LLd: CSR > Pointer** – Irregular jumps propagate to LLd; CSR's bulk accesses raise misses slightly.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	3698775	4299582	526192542
Data refs	1499504	1749817	191806304
I1 misses	4	0	1724
LLi misses	4	0	1690
I1 miss rate	0.00010814	0.00000000	0.00032764
LLi miss rate	0.00010814	0.00000000	0.00032118
D1 misses	674484	502932	1060603
LLd misses	3124	6250	235948
LL misses	3128	6250	237638
D1 miss rate	44.98047354	28.74197702	0.55295524
LLd miss rate	0.20833556	0.35718021	0.12301368
LL miss rate	0.06017376	0.10331605	0.03309727

Table 11: ER Graph: 50,000 nodes and average degree 8

ER Graph (50k, deg 8)

- **D1: Pointer > CSR** – Sparse randomness hurts pointer locality more than CSR's sequential edge layout.
- **LLd: CSR > Pointer** – More edges per fetch in CSR increase LLC traffic despite better D1 behavior.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	12100033	13901182	2005857600
Data refs	5100014	5350417	725784606
I1 misses	4	10	1717
LLi misses	4	10	1710
I1 miss rate	0.00003306	0.00007194	0.00008560
LLi miss rate	0.00003306	0.00007194	0.00008525
D1 misses	2326835	1590139	4150806
LLd misses	763683	81112	3837551
LL misses	763687	81122	3839261
D1 miss rate	45.62409044	29.71990781	0.57190604
LLd miss rate	14.97413537	1.51599399	0.52874516
LL miss rate	4.44002857	0.42137799	0.14054773

Table 12: ER Graph: 50,000 nodes and average degree 32

ER Graph (50k, deg 32)

- **D1: Pointer $\gg$ CSR** – Dense connectivity creates large working sets; CSR significantly reduces misses via locality.
- **LLd: Pointer $\gg$ CSR** – Pointer chasing spreads accesses widely, overwhelming LLd more than CSR.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	7397961	8598676	1056812119
Data refs	2999174	3499458	384376229
I1 misses	4	0	1721
LLi misses	4	0	1714
I1 miss rate	0.00005407	0.00000000	0.00016285
LLi miss rate	0.00005407	0.00000000	0.00016219
D1 misses	1371515	1056855	2117502
LLd misses	294117	40510	1822576
LL misses	294121	40510	1824290
D1 miss rate	45.72975759	30.20053391	0.55089307
LLd miss rate	9.80660008	1.15760784	0.47416460
LL miss rate	2.82886584	0.33484503	0.12658234

Table 13: ER Graph: 100,000 nodes and average degree 8

ER Graph (100k, deg 8)

- **D1: Pointer > CSR** – Large graph size increases working set; CSR maintains better spatial locality.
- **LLd: Pointer ≫ CSR** – Deeper cache suffers from scattered pointer accesses across many nodes.

Metric	Pointer BFS	CSR BFS	CSR Conversion
Instr. refs	2300028	2901743	185638528
Data refs	900011	1300682	70170681
I1 misses	4	17	1696
LLi misses	4	17	1663
I1 miss rate	0.00017391	0.00058585	0.00091360
LLi miss rate	0.00017391	0.00058585	0.00089583
D1 misses	81256	37540	315823
LLd misses	6250	12503	76570
LL misses	6254	12520	78233
D1 miss rate	9.02833410	2.88617817	0.45007829
LLd miss rate	0.69443596	0.96126494	0.10911965
LL miss rate	0.19543512	0.29792322	0.03058256

Table 14: Star Graph: 100,000 nodes

Star Graph (100k nodes)

- **D1: Pointer > CSR** – Central hub causes repeated pointer dereferences; CSR keeps adjacency compact.
- **LLd: CSR > Pointer** – CSR's bulk edge traversal increases LLd accesses slightly, hence increased LLd misses despite fewer D1 misses.

5.2 Varying Associativity

Across all four graph structures, increasing cache associativity from 2-way to 8-way produces minimal change in overall miss behavior, indicating that conflict misses are not the dominant bottleneck in these BFS workloads.

In the chain graph, pointer-based D1 misses remain effectively constant at 75,005 across all associativity levels, while CSR fluctuates only slightly around 33k, showing that the linear traversal pattern already aligns well with cache mapping regardless of associativity.

In the grid graph, pointer misses decrease modestly from 436,384 to 422,784 as associativity increases, while CSR shows a mild increase, suggesting that spatial locality dominates and additional associativity yields diminishing returns once basic conflicts are resolved.

For the ER graph with 50,000 nodes and degree 32, both pointer and CSR miss counts remain nearly unchanged across associativity levels, confirming that the dominant cost arises from capacity and access irregularity rather than set conflicts. Similarly, in the ER graph with 100,000 nodes and degree 8, the variations are numerically small relative to total misses, reinforcing the conclusion that BFS memory behavior is largely insensitive to associativity beyond baseline levels. Overall, the data shows that representation choice (pointer vs CSR) drives cache performance far more strongly than associativity tuning in graph traversal workloads.

Associativity	**D1 Misses**		**LLd Misses**	
	Pointer	**CSR**	**Pointer**	**CSR**
2-way	75005	33599	6250	12510
4-way	75005	35012	6250	12503
8-way	75005	31288	6250	12503

Table 15: Chain Graph : 100,000 Nodes

Associativity	**D1 Misses**		**LLd Misses**	
	Pointer	**CSR**	**Pointer**	**CSR**
2-way	436384	278745	6240	12483
4-way	430885	288652	6240	12483
8-way	422784	297692	6240	12483

Table 16: Grid Graph : 316 x 316 (around 100,000 Nodes)

Associativity	**D1 Misses**		**LLd Misses**	
	Pointer	**CSR**	**Pointer**	**CSR**
2-way	2326421	1590149	763708	81112
4-way	2326226	1589536	763703	81112
8-way	2326835	1590139	763683	81112

Table 17: ER Graph: 50,000 nodes and average degree 32

Associativity	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
2-way	1371894	1057722	294162	40510
4-way	1371632	1056952	294155	40510
8-way	1371515	1056855	294117	40510

Table 18: ER Graph: 100,000 nodes and average degree 8

5.3 Varying Cache Size

Changing cache size from 16 KB to 64 KB produces more visible improvements than associativity changes, but the gains vary significantly by graph structure and memory footprint.

In the chain graph, both pointer and CSR D1 miss counts remain nearly identical across all cache sizes, indicating that the streaming access pattern exhibits predictable locality and does not benefit substantially from additional capacity.

In contrast, the grid graph shows a clear reduction in misses for both representations as cache size increases, with CSR dropping from 344,567 to 252,662 D1 misses, demonstrating that larger caches better accommodate the two-dimensional frontier expansion characteristic of grid traversal.

The effect is even stronger in the ER graph with 50,000 nodes and degree 32, where pointer misses fall from roughly 2.41 million to 2.17 million and CSR from 1.71 million to 1.36 million, highlighting the sensitivity of dense random graphs to cache capacity due to their large working sets. Finally, in the ER graph with 100,000 nodes and degree 8, both representations again benefit from increased cache size, though the improvement is more gradual, reflecting a balance between sparse connectivity and large vertex count. Taken together, the results show that cache capacity has a measurable but workload-dependent impact, with the largest benefits appearing in graphs that generate broad frontiers and high edge counts.

Cache Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
16 KB	75005	31295	6250	12510
32 KB	75005	31295	6250	12510
64 KB	75005	31288	6250	12503

Table 19: Chain Graph : 100,000 Nodes

Cache Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
16 KB	435678	344567	6240	12483
32 KB	422784	297692	6240	12483
64 KB	396365	252662	6240	12483

Table 20: Grid Graph : 316 x 316 (around 100,000 Nodes)

Cache Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
16 KB	2409730	1705384	763700	81112
32 KB	2326835	1590139	763683	81112
64 KB	2165384	1361771	763649	81112

Table 21: ER Graph: 50,000 nodes and average degree 32

Cache Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
16 KB	1392110	1083277	294151	40510
32 KB	1371515	1056855	294117	40510
64 KB	1331727	1005093	294052	40510

Table 22: ER Graph: 100,000 nodes and average degree 8

5.4 Varying Line Size

Changing the cache line size from 32 B to 128 B consistently reduces miss counts, though the impact varies by graph structure. In the chain graph, both pointer and CSR D1 misses drop sharply (e.g., pointer from 150,000 to 37,505), showing that the sequential access pattern benefits greatly from increased spatial locality.

The grid graph also sees steady improvements, with pointer misses decreasing from 621,202 to 335,112 and CSR from 308,434 to 246,258, reflecting moderate locality in its two-dimensional traversal.

The effect is more pronounced in the ER graph with 50,000 nodes and degree 32, where pointer misses fall from about 3.02 million to 1.71 million and CSR from 1.50 million to 1.29 million, highlighting how larger cache lines help capture scattered edge accesses in dense graphs. In the ER graph with 100,000 nodes and degree 8, both representations again improve, though more gradually, indicating a balance between sparsity and working set size.

Overall, larger cache lines improve performance by exploiting spatial locality, with the greatest gains in structured or dense access patterns.

Line Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
32 B	150006	62549	6250	12503
64 B	75005	31288	6250	12503
128 B	37505	15654	3125	6252

Table 23: Chain Graph : 100,000 Nodes

Line Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
32 B	621202	308434	6240	12483
64 B	415721	263583	6240	12483
128 B	335112	246258	39566	15647

Table 24: Grid Graph : 316 x 316 (around 100,000 Nodes)

Line Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
32 B	3019553	1499685	763664	81112
64 B	2163716	1361021	763664	81112
128 B	1708948	1291866	409972	43709

Table 25: ER Graph: 50,000 nodes and average degree 32

Line Size	D1 Misses		LLd Misses	
	Pointer	CSR	Pointer	CSR
32 B	1694579	1086034	294082	40510
64 B	1331546	1005082	294079	40510
128 B	1131775	965128	197286	51541

Table 26: ER Graph: 100,000 nodes and average degree 8

6 Conclusion

This study examined the performance implications of two graph representations ; pointer-based adjacency lists and Compressed Sparse Row (CSR) ; for Breadth-First Search (BFS), evaluated across runtime, cache behavior, and varying cache configurations.

Runtime Comparison

CSR consistently outperforms pointer-based BFS across all tested graph topologies. The speedup ranges from a modest **1.18**× on a chain graph (100k nodes) to a substantial **7.04**× on a dense random graph (50k nodes, degree 32). Beyond raw throughput, CSR also exhibits significantly lower execution variance, indicating more predictable and stable performance due to deterministic memory access patterns.

Cache Behavior

CSR generates substantially fewer **D1 cache misses** than the pointer-based implementation across all graph types. For example, in the dense ER graph (50k nodes, degree 32), pointer BFS incurs approximately 2.33 million D1 misses versus 1.59 million for CSR. However, CSR tends to produce slightly more **LLd (last-level data) cache misses** in

sparse graphs ; a consequence of its bulk sequential access patterns loading more contiguous data into the deeper cache hierarchy. In dense graphs, the trend reverses dramatically, with pointer BFS generating far more LLd misses due to widespread pointer chasing that scatters accesses across memory.

Memory Layout and Spatial Locality

The performance advantage of CSR stems from its contiguous memory layout. Pointer-based adjacency lists require *pointer chasing* ; each neighbor access involves dereferencing a heap-allocated `Edge` node, which may reside at an arbitrary memory location, leading to poor spatial locality and frequent cache misses. In contrast, CSR stores all neighbor indices in a flat `col_idx` array, enabling sequential memory access. This allows the hardware prefetcher to anticipate upcoming addresses and preload data before it is needed, reducing memory stall cycles. With a 64-byte cache line, CSR can pack the adjacency list of an average-degree-8 vertex into a single or few cache lines, processing multiple edges per miss ; an efficiency impossible with fragmented pointer structures.

Effect of Varying Cache Parameters

Cache Size. Increasing cache size from 16 KB to 64 KB yields measurable but workload-dependent improvements. Structured graphs (e.g., grid) and dense random graphs benefit more noticeably ; CSR D1 misses on the 316×316 grid drop from 344,567 to 252,662 ; while the chain graph shows virtually no change due to its already-streaming access pattern. Both representations benefit, but the relative advantage of CSR is maintained across all sizes.

Associativity. Varying associativity from 2-way to 8-way has negligible effect on miss counts for both implementations. Changes are numerically small relative to total misses, confirming that *conflict misses are not the dominant bottleneck* in BFS workloads. The data layout choice (pointer vs. CSR) has a far larger impact on performance than associativity tuning.

Cache Line Size. Increasing cache line size from 32 B to 128 B consistently reduces D1 miss counts for both implementations, but CSR benefits proportionally more. For the chain graph, pointer D1 misses drop from 150,006 to 37,505 and CSR from 62,549 to 15,654 ; a near 4× reduction for both ; while for denser graphs the benefit is more pronounced for CSR. This is because CSR's contiguous edge arrays are better positioned to exploit the increased spatial locality that larger lines provide: a wider cache line captures more consecutive neighbor indices in a single fetch. Pointer-based structures, with their scattered heap allocations, cannot fully utilize larger cache lines since the neighboring data is not co-located in memory.

Overall. Across all experiments, the choice of graph representation is the single most impactful factor in BFS cache and runtime performance. CSR's contiguous memory layout delivers consistent speedups, lower variance, and superior cache utilization ; benefits that grow with graph size and edge density.